

ICS1513 - Computer Networks Laboratory

Experiment 3: Implementation of HTTP Client using TCP Sockets

Nitin H - 3122247001042

July 22 2026

Aim

To emulate an HTTP client using a raw TCP socket, performing DNS lookup / hostname resolution of a URL, connecting to a remote or local server, and downloading the returned HTTP response, separating headers from the payload and saving the payload to file.

Question Summary

- **Mandatory Program (a):** Develop a TCP socket-based HTTP client in C that accepts a URL from the user, connects to the corresponding web server, sends an HTTP GET request, retrieves the requested webpage/file, and stores the downloaded content locally.
- **Mandatory Program (b):** Develop an HTTP client using TCP sockets in C to communicate with a Python-based web server (`python3 -m http.server`) and download web resources (HTML, text, images, PDF files) served from a local directory (`CN_Web_Server`).
- In both cases, the client must analyze the performance of the communication by measuring response time, total download time, data size, and throughput.
- **Advanced Program (optional):** Develop a secure HTTPS client in C using TCP sockets and OpenSSL, establishing a TCP connection to an HTTPS server (port 443), performing an SSL/TLS handshake, validating the server certificate, sending a GET request securely, and analyzing security/performance parameters (TCP connection time, SSL handshake time, data transfer rate, RTT).

1 Emulating DNS Lookup and Fetching a Remote URL

Algorithm

1. Prompt the user to enter a URL and read it into a buffer.
2. Parse the URL using `parseURL()`:
 - (a) Skip past the "`http://`" or "`https://`" scheme prefix if present.
 - (b) Locate the first `'/'` after the scheme to separate the hostname from the path.
 - (c) If no `'/'` is found, the path defaults to `"/`" and the entire remaining string is the hostname.
 - (d) Copy out the hostname and path into separate buffers.
3. Resolve the hostname to an IP address using `getaddrinfo()` with `hints.ai_family = AF_INET` and `hints.ai_socktype = SOCK_STREAM`, querying port 80.
4. Create a TCP socket using `socket()` with the family/socketype/protocol returned by `getaddrinfo()`.

5. Connect to the resolved address using `connect()`, then free the `addrinfo` structure with `freeaddrinfo()`.
6. Construct an HTTP/1.1 GET request string with `Host` and `Connection: close` headers, and send it using `send()`.
7. Open an output file (`dc.txt`) for writing in binary mode.
8. Loop: receive data using `recv()`.
 - (a) On the first chunk received, locate the header/body delimiter ("`\r\n\r\n`"), print the headers to the console, and write the remaining body bytes (if any) to the output file.
 - (b) On subsequent chunks, write all received bytes directly to the output file.
9. Close the output file and the socket.

Code (Q31.c)

```

1  #define _XOPEN_SOURCE 700
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <unistd.h>
6  #include <netdb.h>
7  #include <sys/socket.h>
8
9  #define BUFFER_SIZE 4096
10
11 void parseURL(char *url, char *hostname, char *path)
12 {
13     strcpy(path, "/");
14
15     char *url_ptr = url;
16     if (strncmp(url_ptr, "http://", 7) == 0)
17     {
18         url_ptr += 7;
19     }
20     else if (strncmp(url_ptr, "https://", 8) == 0)
21     {
22         url_ptr += 8;
23     }
24
25     char *slash = strchr(url_ptr, '/');
26     if (slash != NULL) {
27         int host_len = slash - url_ptr;
28         strncpy(hostname, url_ptr, host_len);
29         hostname[host_len] = '\0';
30         strcpy(path, slash);
31     } else {
32         strcpy(hostname, url_ptr);
33     }
34 }
35
36 int main()
37 {
38     char url[512];
39     char hostname[256];
40     char path[256];
41
42     printf("Enter URL: ");
43     scanf("%511s", url);
44
45     parseURL(url, hostname, path);
46
47     printf("Hostname: %s\n", hostname);
48     printf("Path: %s\n", path);

```

```

49
50 struct addrinfo hints, *res;
51 memset(&hints, 0, sizeof(hints));
52 hints.ai_family = AF_INET;
53 hints.ai_socktype = SOCK_STREAM;
54
55 int status = getaddrinfo(hostname, "80", &hints, &res);
56 if (status != 0) {
57     fprintf(stderr, "DNS_Error: %s\n", gai_strerror(status));
58     return 1;
59 }
60
61 int client_socket = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
62 if (client_socket < 0) {
63     perror("Socket_creation_failed");
64     freeaddrinfo(res);
65     return 1;
66 }
67
68 if (connect(client_socket, res->ai_addr, res->ai_addrlen) < 0) {
69     perror("Connection_failed");
70     close(client_socket);
71     freeaddrinfo(res);
72     return 1;
73 }
74
75 freeaddrinfo(res);
76
77 char request[BUFFER_SIZE];
78 snprintf(request, sizeof(request),
79     "GET_%s_HTTP/1.1\r\n"
80     "Host: %s\r\n"
81     "User-Agent: Raw-C-Client\r\n"
82     "Connection: close\r\n\r\n",
83     path, hostname);
84
85 if (send(client_socket, request, strlen(request), 0) < 0) {
86     perror("Failed_to_send_HTTP_request");
87     close(client_socket);
88     return 1;
89 }
90
91 FILE *output_file = fopen("dc.txt", "wb");
92 if (!output_file) {
93     perror("Failed_to_open_output_file");
94     close(client_socket);
95     return 1;
96 }
97
98 char buffer[BUFFER_SIZE];
99 ssize_t bytes_received;
100 int header_parsed = 0;
101
102 while ((bytes_received = recv(client_socket, buffer, BUFFER_SIZE - 1, 0)) > 0) {
103     buffer[bytes_received] = '\0';
104     if (!header_parsed) {
105         char *body_start = strstr(buffer, "\r\n\r\n");
106         if (body_start != NULL) {
107             body_start += 4;
108
109             int header_len = body_start - buffer;
110             printf("%.s", header_len, buffer);
111             printf("\nBody_Content_Saved\n");
112
113             size_t body_bytes = bytes_received - header_len;
114             if (body_bytes > 0) {
115                 fwrite(body_start, 1, body_bytes, output_file);
116             }

```

```

117         header_parsed = 1;
118     } else {
119         printf("%s", buffer);
120     }
121 } else {
122     fwrite(buffer, 1, bytes_received, output_file);
123 }
124 }
125
126 if (bytes_received < 0) {
127     perror("Error reading from socket");
128 }
129
130 fclose(output_file);
131 close(client_socket);
132
133 return 0;
134 }

```

Sample Input and Output

Input: http://google.com

Output:

Hostname: google.com

Path: /

HTTP/1.1 302 Found

Location: http://www.google.com/sorry/index?continue=...

Date: Thu, 23 Jul 2026 04:06:02 GMT

Pragma: no-cache

Cache-Control: no-store, no-cache, must-revalidate

Content-Type: text/html; charset=UTF-8

Server: scaffolding on HTTPServer2

Content-Length: 348

X-XSS-Protection: 0

Connection: close

Body Content Saved

```
Body Content Saved
● cseal@wtl-13:~/ICS1513-Computer-Networks-Labratory/Experiment 3$ ./a.out
Enter URL: http://google.com
Hostname: google.com
Path: /

HTTP/1.1 302 Found
Location: http://www.google.com/sorry/index?continue=http://google.com/&q=EgS2Ssx6GKqmht
MGiil7rPySNeXbBPYNF0mr5B0eCcQmz9rWryQyBil82hzkcWiVPbM28FIPjICclJaAUM
Date: Thu, 23 Jul 2026 04:06:02 GMT
Pragma: no-cache
Expires: Fri, 01 Jan 1990 00:00:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Content-Type: text/html; charset=UTF-8
Server: scaffolding on HTTPServer2
Content-Length: 348
X-XSS-Protection: 0
Connection: close

Body Content Saved
● cseal@wtl-13:~/ICS1513-Computer-Networks-Labratory/Experiment 3$ cat dc.txt
<HTML><HEAD><meta http-equiv="content-type" content="text/html; charset=utf-8">
<TITLE>302 Moved</TITLE></HEAD><BODY>
<H1>302 Moved</H1>
The document has moved
<A HREF="http://www.google.com/sorry/index?continue=http://google.com/&q=EgS2Ssx6GKq
mhtMGiil7rPySNeXbBPYNF0mr5B0eCcQmz9rWryQyBil82hzkcWiVPbM28FIPjICclJaAUM">here</A>.
</BODY></HTML>
○ cseal@wtl-13:~/ICS1513-Computer-Networks-Labratory/Experiment 3$
```

Figure 1: Output of Q31.c – HTTP request/response headers and saved body content (dc.txt)

2 HTTP Client on a Personal Python Web Server

Algorithm

1. Start a local HTTP server using Python's built-in `http.server` module on port 8000, serving the current directory (containing `index.html`).
2. Prompt the user to enter a URL (e.g. `http://localhost:8000/index.html`) and parse it with `parseURL()` to extract the hostname, port, and path, defaulting to port 8000.
3. Resolve the hostname using `gethostbyname()`.
4. Create a TCP socket and `connect()` to the resolved server address and port.
5. Build and send an HTTP/1.1 GET request with `Host` and `Connection: close` headers using `send()`, recording the start timestamp with `clock_gettime()`.
6. Determine the output filename from the requested path (or default to `downloaded_index.html`), and open it for writing in binary mode.
7. Loop: receive data with `recv()`.
 - (a) On the first byte received, record the response time (time-to-first-byte).
 - (b) Split headers from the payload at `"\r\n\r\n"`, writing only the payload portion to the output file.
 - (c) For subsequent chunks, write all bytes directly to the file.
8. Record the end timestamp, close the file and socket.
9. Compute and display performance metrics: response time, total download time, and throughput (KB/s).

Code (Q32.c)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <time.h>
6  #include <sys/socket.h>
7  #include <arpa/inet.h>
8  #include <netdb.h>
9
10 #define BUFFER_SIZE 8192
11
12 // Helper function to convert timespec difference to milliseconds
13 double get_time_ms(struct timespec start, struct timespec end) {
14     return (end.tv_sec - start.tv_sec) * 1000.0 +
15           (end.tv_nsec - start.tv_nsec) / 1000000.0;
16 }
17
18 // Fixed and complete URL Parsing Function
19 void parseURL(char *url, char *hostname, int *port, char *path)
20 {
21     strcpy(path, "/");
22     *port = 8000; // Defaulting to port 8000 for your Python server
23
24     char *url_ptr = url;
25     if (strncmp(url_ptr, "http://", 7) == 0)
26     {
27         url_ptr += 7;
28     }
29     else if (strncmp(url_ptr, "https://", 8) == 0)
```

```

30     {
31         url_ptr += 8;
32     }
33
34     char *slash = strchr(url_ptr, '/');
35     if (slash != NULL) {
36         int host_len = slash - url_ptr;
37         strncpy(hostname, url_ptr, host_len);
38         hostname[host_len] = '\0';
39         strcpy(path, slash);
40     } else {
41         strcpy(hostname, url_ptr);
42     }
43
44     // Extract port from hostname if ':' is present (e.g., "localhost:8000")
45     char *colon = strchr(hostname, ':');
46     if (colon != NULL) {
47         *port = atoi(colon + 1);
48         *colon = '\0'; // Truncate hostname to separate it from the port
49     }
50 }
51
52 int main()
53 {
54     char url[512];
55     char hostname[256];
56     char path[256];
57     int port;
58
59     printf("Enter URL: ");
60     if (scanf("%511s", url) != 1) {
61         fprintf(stderr, "Invalid input.\n");
62         return 1;
63     }
64
65     parseURL(url, hostname, &port, path);
66
67     printf("\n--- Parsed Details ---\n");
68     printf("Hostname: %s\n", hostname);
69     printf("Port: %d\n", port);
70     printf("Path: %s\n", path);
71
72     struct timespec start_total, start_resp, end_resp, end_total;
73     clock_gettime(CLOCK_MONOTONIC, &start_total);
74
75     // 1. Resolve Hostname to IP Address
76     struct hostent *server = gethostbyname(hostname);
77     if (server == NULL) {
78         fprintf(stderr, "Error: Could not resolve host %s\n", hostname);
79         return 1;
80     }
81
82     // 2. Create Socket
83     int sockfd = socket(AF_INET, SOCK_STREAM, 0);
84     if (sockfd < 0) {
85         perror("Error opening socket");
86         return 1;
87     }
88
89     // 3. Setup Server Address
90     struct sockaddr_in serv_addr;
91     memset(&serv_addr, 0, sizeof(serv_addr));
92     serv_addr.sin_family = AF_INET;
93     serv_addr.sin_port = htons(port);
94     memcpy(&serv_addr.sin_addr.s_addr, server->h_addr_list[0], server->h_length);
95
96     // 4. Connect to Server
97     if (connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {

```

```

98     perror("Error connecting to server");
99     close(sockfd);
100     return 1;
101 }
102
103 // 5. Build and Send HTTP GET Request
104 char request[1024];
105 snprintf(request, sizeof(request),
106     "GET_%s_HTTP/1.1\r\n"
107     "Host:_%s:%d\r\n"
108     "Connection:_close\r\n\r\n",
109     path, hostname, port);
110
111 clock_gettime(CLOCK_MONOTONIC, &start_resp);
112 send(sockfd, request, strlen(request), 0);
113
114 // Determine output file name
115 char *filename = strrchr(path, '/');
116 char out_filename[300];
117 if (filename && strlen(filename) > 1) {
118     snprintf(out_filename, sizeof(out_filename), "downloaded_%s", filename + 1);
119 } else {
120     strcpy(out_filename, "downloaded_index.html");
121 }
122
123 FILE *fp = fopen(out_filename, "wb");
124 if (!fp) {
125     perror("Error creating file");
126     close(sockfd);
127     return 1;
128 }
129
130 // 6. Receive Data & Separate Headers from Payload
131 char buffer[BUFFER_SIZE];
132 ssize_t bytes_received;
133 size_t total_payload_bytes = 0;
134 int headers_ended = 0;
135
136 while ((bytes_received = recv(sockfd, buffer, sizeof(buffer), 0)) > 0) {
137     if (!headers_ended) {
138         // First byte returned -> measure response time (TTFB)
139         clock_gettime(CLOCK_MONOTONIC, &end_resp);
140
141         // Find the delimiter (\r\n\r\n) between headers and payload
142         char *header_end_ptr = strstr(buffer, "\r\n\r\n");
143         if (header_end_ptr) {
144             headers_ended = 1;
145             header_end_ptr += 4; // Skip past "\r\n\r\n"
146
147             size_t header_len = header_end_ptr - buffer;
148             size_t payload_len = bytes_received - header_len;
149
150             if (payload_len > 0) {
151                 fwrite(header_end_ptr, 1, payload_len, fp);
152                 total_payload_bytes += payload_len;
153             }
154         }
155     } else {
156         // Write remaining raw binary payload
157         fwrite(buffer, 1, bytes_received, fp);
158         total_payload_bytes += bytes_received;
159     }
160 }
161
162 clock_gettime(CLOCK_MONOTONIC, &end_total);
163 fclose(fp);
164 close(sockfd);
165

```

```

166 // 7. Calculate Performance Metrics
167 double response_time = get_time_ms(start_resp, end_resp);
168 double total_time_ms = get_time_ms(start_total, end_total);
169 double total_time_sec = total_time_ms / 1000.0;
170
171 double throughput_kbps = (total_time_sec > 0) ?
172     ((double)total_payload_bytes / 1024.0) / total_time_sec : 0.0;
173
174 // Output Performance Analysis
175 printf("=====\n");
176 printf("PERFORMANCE METRICS SUMMARY\n");
177 printf("=====\n");
178 printf("Saved File Name: %s\n", out_filename);
179 printf("Data Size Saved: %zu bytes (%.2f KB)\n", total_payload_bytes, total_payload_bytes / 1024.0);
180 printf("Response Time: %.3f ms\n", response_time);
181 printf("Total Download: %.3f ms\n", total_time_ms);
182 printf("Throughput: %.2f KB/s\n", throughput_kbps);
183 printf("=====\n");
184
185 return 0;
186 }

```

Sample Input and Output

Input: http://localhost:8000/index.html

Output:

```

Hostname : localhost
Port      : 8000
Path      : /index.html

```

```

=====
PERFORMANCE METRICS SUMMARY
=====
Saved File Name   : downloaded_index.html
Data Size Saved   : 167 bytes (0.16 KB)
Response Time     : 3.482 ms
Total Download    : 3.658 ms
Throughput        : 44.58 KB/s
=====

```

Learning Outcomes

- Learnt to initialize and serve content from a Python-based local HTTP server.
- Learnt to implement DNS lookup / hostname resolution (both `getaddrinfo()` and `gethostbyname()`) as part of an HTTP client.
- Learnt to implement client-side download of files over raw TCP sockets, including separating HTTP headers from the response payload and measuring response time, total download time, and throughput.

```
cseal@wtl-13:~/ICS1513-Computer-Networks-Labratory/Experiment 3/CN_Web_Server$ cc Q32.c
cseal@wtl-13:~/ICS1513-Computer-Networks-Labratory/Experiment 3/CN_Web_Server$ python3
m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [23/Jul/2026 09:38:43] "GET /index.html HTTP/1.1" 200 -
[]

cseal@wtl-13:~/ICS1513-Computer-Networks-Labratory/Experiment 3/CN_Web_Server$ cc Q32.c
cseal@wtl-13:~/ICS1513-Computer-Networks-Labratory/Experiment 3/CN_Web_Server$ ./a.out
Enter URL: http://localhost:8000/index.html

--- Parsed Details ---
Hostname : localhost
Port      : 8000
Path      : /index.html

=====
PERFORMANCE METRICS SUMMARY
=====
Saved File Name   : downloaded_index.html
Data Size Saved   : 167 bytes (0.16 KB)
Response Time     : 3.482 ms
Total Download    : 3.658 ms
Throughput        : 44.58 KB/s
=====

cseal@wtl-13:~/ICS1513-Computer-Networks-Labratory/Experiment 3/CN_Web_Server$ cat index.html
<html>
<head>
<title>Computer Networks Lab</title>
</head>
<body>
<h1>Welcome to CN Lab</h1>
<p>This webpage is downloaded using TCP socket client.</p>
</body>
</html>
cseal@wtl-13:~/ICS1513-Computer-Networks-Labratory/Experiment 3/CN_Web_Server$
```

Figure 2: Output of Q32.c – Python HTTP server log, client performance metrics, and downloaded file contents